

Ex74_KalmanBodyGravitation

April 24, 2026

1 MSE StatDig : Chap 7 “Optimum filter”

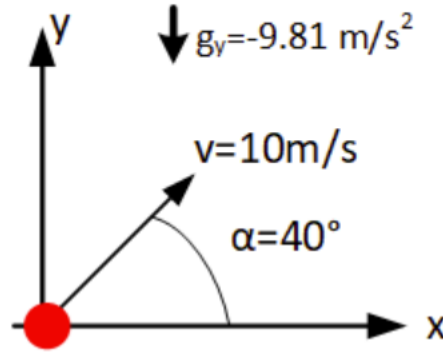
1.1 Ex 7.4 Kalman filter - Body in gravitation

ver : DLY/24.04.2026

1.2 General

1.2.1 Description

Let's assume that a body is ejected in the air with a constant velocity $v = 10$ [m/s] at a given angle $\alpha = 40\text{degrees}$



The goal is to estimate the trajectory (position(x,y)). The position (x,y) of the ball across time is measured but the measurement is very noisy. The goal is to filter the noise with the Kalman filter.

The dynamic model of the system is given by the following equations:

$$\begin{cases} x[k] = x[k-1] + v_x[k-1] \cdot \Delta t \\ v_x[k] = v_x[k-1] \\ y[k] = y[k-1] + v_y[k-1] \cdot \Delta t + \frac{1}{2}a_y[k-1]\Delta t^2 \\ v_y[k] = v_y[k-1] + a_y[k-1] \cdot \Delta t \\ a_y[k] = a_y[k-1] \end{cases}$$

where the acceleration is the gravity $a_y[k] = -g = -9.81$.

Those equations can be written in matrix shape:

$$\begin{bmatrix} x[k] \\ v_x[k] \\ y[k] \\ v_y[k] \\ a_y[k] \end{bmatrix} = \begin{bmatrix} 1 & \Delta t & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x[k-1] \\ v_x[k-1] \\ y[k-1] \\ v_y[k-1] \\ a_y[k-1] \end{bmatrix}$$

And finally in the compact form:

$$X[k] = F \cdot X[k-1]$$

Where $\mathbf{F}$ is the transition matrix and $\mathbf{X}[k]$ is the current state, and $\mathbf{X}[k-1]$ is the previous state.

- The position (x, y) of the ball is measured in the presence of a disturbance $\mathbf{vn}[k]$, where the noise is normally distributed with unit variance $\mathcal{N}(0, R)$, where:

$$R = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix}$$

The measurement equation becomes:

$$\begin{bmatrix} z_x[k] \\ z_y[k] \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x[k] \\ v_x[k] \\ y[k] \\ v_y[k] \\ a_y[k] \end{bmatrix} + \mathbf{vn}[k]$$

And finally, in compact form:

$$z[k] = H \cdot X[k] + \mathbf{vn}[k]$$

- To generate the trajectory, the initial state also has to be defined:

$$X[0] = \begin{bmatrix} x[0] \\ v_x[0] \\ y[0] \\ v_y[0] \\ a_y[0] \end{bmatrix} = \begin{bmatrix} 0 \\ v \cdot \cos(40^\circ) \\ 0 \\ v \cdot \sin(40^\circ) \\ -g \end{bmatrix}$$

1.2.2 Work

Ex1 : Signal generation

- Generate the trajectory with the following code and display the trajectory and the measured trajectory:

```

g = 9.81      # [m/s^2] gravitation
v = 10        # [m/s] initial speed
dt = 2e-3     # [s] sampling period
alpha = 40    # [°] ejection angle
SigmaV = 0.2  # [m] standard deviation of the measurement noise
tmax = 2      # [s] Duration of the measurement
N = int(tmax//dt) # [-] Number of samples
n = np.arange(N)    # [-] numerical index
alpha = 40 # [°] initial angle
# measurement noise
np.random.seed(0)
noiseX = SigmaV * np.random.normal(0,1,N)
noiseY = SigmaV * np.random.normal(0,1,N)

F = np.array([[1,dt,0,0,0],
              [0,1 ,0,0,0],
              [0,0 ,1 ,dt,1/2*dt**2],
              [0,0,0,1 ,dt],
              [0,0,0,0,1]]) # Transition state matrix (A)
G = 0 # Input dynamic matrix (B)
H = np.array([[1,0,0,0,0],
              [0,0,1,0,0]]) # Observation matrix (C)

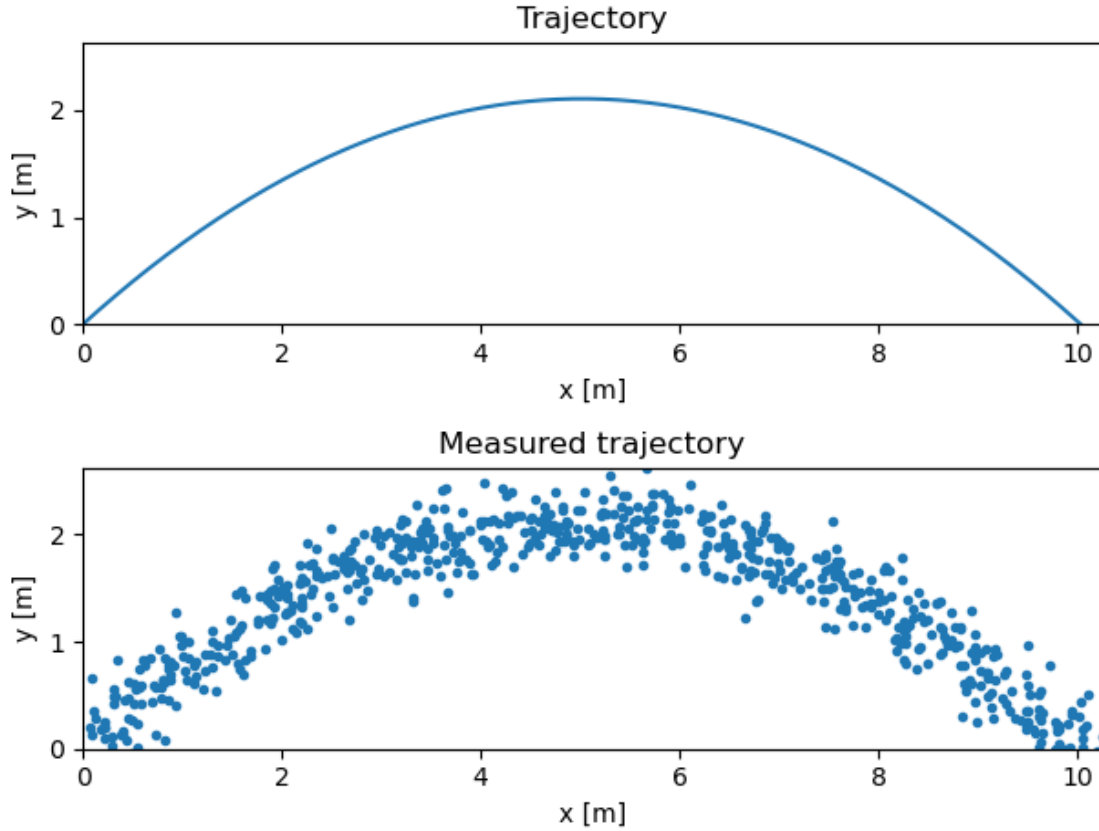
xHatInit = [0,v*np.cos(alpha/180*np.pi),0,v*np.sin(alpha/180*np.pi),-g] # initial state

# Trajectory generation
# generate the trajectory
X = np.zeros((len(xHatInit),N));
X[:,0] = xHatInit;
for k in range(1,N):
    X[:,k] = np.dot(F,X[:,k-1])

x = X[0,:]; # get x position
y = X[2,:]; # get y position

# Add measurement noise
xnoise = x + noiseX;
ynoise = y + noiseY;

```



Ex2 : Kalman filter

Ex2.1 Kalman parameters The Kalman filter needs the following parameters:

- F : Transition Matrix $[p \times p]$
- H : Measurement Matrix $[q \times p]$
- $x(0|0)$: Initial state $[p \times 1]$
- $z[k]$: Measurement matrix $[q \times 1]$
- Q_w : Covariance matrix process noise $[p \times p]$
- Q_v : Covariance matrix measurement noise $[q \times q]$
- $P(0|0)$: Covariance matrix error $[p \times p]$

where $p=5$ is the number of state and $q=2$ is the number of measurement. Each of those matrices are known and were already defined during the trajectory generation except Q_w , Q_v and $P(0|0)$.

- Build the remaining matrices :

```
Qv= [[0.04 0. ]
      [0.   0.04]]
```

```
Qw= 0
```

```
P(0|0)= [[1. 0. 0. 0. 0.]
          [0. 1. 0. 0. 0.]
```

```
[0. 0. 1. 0. 0.]
[0. 0. 0. 1. 0.]
[0. 0. 0. 0. 1.]]
```

Ex2.2 Kalman recursivity The following matrices are necessary to create the Kalman recursion, where the following matrices are computed:

- $\hat{X}[k|k-1]$: State estimation at sample k based on the measurement up to k-1 [$p \times 1$]
- $P[k|k-1]$: Error covariance at sample k based on the measurement up to k-1 [$p \times p$]
- $K[k]$: Kalman gain computed by minimizing the innovation (noise) on the measurement up to k [$p \times q$]
- $P[k|k]$: Error covariance at sample k based on the measurement up to k [$p \times p$]
- $\hat{X}[k|k]$: State estimation at sample k based on the measurement up to k [$p \times 1$]

The calculation can be performed in two steps within a loop over each measured sample:

- **Prediction** (based on measurement samples from 1 to n-1):

$$\hat{X}[k|k-1] = F \cdot \hat{X}[k-1|k-1]$$

$$P[k|k-1] = F \cdot P[k-1|k-1] \cdot F^T + Q_w$$

- **Update** (based on n measurement samples and the innovation error):

$$K[k] = P[k|k-1] \cdot H^T \cdot (H \cdot P[k|k-1] \cdot H^T + Q_v)^{-1}$$

$$P[k|k] = (I - K[k] \cdot H) \cdot P[k|k-1]$$

$$\hat{X}[k|k] = \hat{X}[k|k-1] + K[k] \cdot [z[k] - H \cdot \hat{X}[k|k-1]]$$

Your work :

- Build the function **KalmanGain** with **inputs**: F, H, Qw, Qv, x(0|0), P(0|0), z[k] and with **outputs**: K, X[k|k], P[k|k], X[k|k-1], P[k|k-1].
- Compute the the filtered trajectory using the Kalman filter and display the result.
- Display the Kalman gain as well the error covariance matrix accross time

```
def KalmanGain(F, H, Qw, Qv, xHatInit, PInit, z):
    """
    Implements the stationary Kalman filter with:
    State equation      : x[n] = F * x[n-1] + w[n]
    Measurement equation : z[n] = H * x[n]   + v[n]
```

Parameters:

- F : $[p \times p]$ State transition matrix
- H : $[q \times p]$ Measurement matrix
- Qw : $[p \times p]$ Covariance matrix of process noise
- Qv : $[q \times q]$ Covariance matrix of measurement noise
- x_{HatInit} : $[p \times 1]$ Initial state estimate
- P_{Init} : $[p \times p]$ Initial error covariance
- z : $[q \times N]$ Measurement matrix (each column is a measurement vector)

Returns:

- K : $[p \times q \times N]$ Kalman gain at each step
 - x_{Hat} : $[p \times N]$ State estimation at each step
 - P : $[p \times p \times N]$ Error covariance at each step
 - $Pn1$: $[p \times p \times N]$ Error covariance at step k based on $k-1$
 - x_{Hatn1} : $[p \times N]$ State prediction before measurement update
- """

your code here

return K, xHat, P, Pn1, xHatn1

